

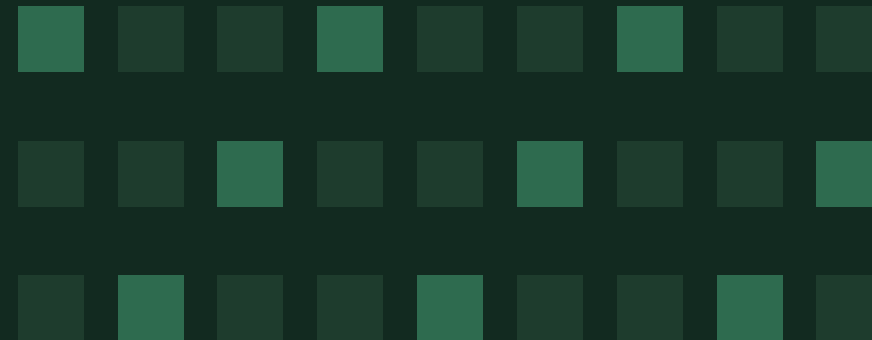
ENTREGA FINAL · ARQUITETURA ORIENTADA A EVENTOS · PUC MINAS ·
2026/2

Rastreabilidade do gado de corte por eventos

Quatro serviços, seis eventos de negócio e um broker. Do cadastro do animal ao embarque no frigorífico — com event sourcing, saga coreografada de compensação e DLQ permanente.

Equipe 04

Paulo Cidrão Gomes Torres (líder) · João Almeida Barbosa Júnior · João Pedro Schlindwein · João Pedro Correia Barros
Matheus Chaves Ferreira · Rafael Corrêa Zart



O que está sendo entregue

Cinco partes técnicas e o fechamento com a cobertura de cada etapa da disciplina.

01	O domínio e a arquitetura	Por que eventos, os quatro critérios e o desenho dos quatro serviços.
02	Contratos e entrega	CloudEvents, chave de partição, compatibilidade e o consumidor idempotente.
03	Agregação por janela	Peso médio por minuto, event time e o que acontece num replay.
04	Event sourcing	O agregado Lote: event store, projeção descartável e reconstrução por replay.
05	Resiliência · projeto final	Saga coreografada de compensação e DLQ permanente com reprocessamento.
✓	Cobertura e evidência	O que cada aula pediu, onde está no repositório e os 26 testes que rodam.

01

PARTE 01

O domínio e a arquitetura

Um processo real de confinamento — e por que ele pede eventos, não chamadas síncronas.

- | O problema: informação espalhada entre cinco setores
- | Os quatro critérios do domínio (ADR-002)
- | Quatro serviços, seis eventos, um broker

O problema: informação espalhada entre setores

Recepção, zootecnia, sanidade, comercial e expedição coordenam por planilha e telefone — sem registro único e ordenado do que aconteceu com cada animal.

SINTOMA

Decisão sem dado confiável

Vender e embarcar depende do peso mais recente, do status vacinal e do lote atual — três números que vivem em lugares diferentes.

IMPACTO

Custo direto, não abstrato

Cada atraso vira animal parado a mais no confinamento; cada erro vira embarque recusado na triagem do frigorífico.

ORIGEM

Domínio com vivência real

Trazido por integrante com rotina de fazenda de confinamento: pesagem, formação de lote e negociação com frigorífico.

A PERGUNTA QUE O SISTEMA RESPONDE

“O que é verdade sobre este animal (ou este lote) agora — e dá para provar que aconteceu na ordem em que realmente aconteceu?”

O que acontece com o animal é uma sequência de fatos passados: foi pesado, foi vacinado, foi rejeitado. Registrar cada fato como evento imutável num log ordenado dá às duas pontas o que falta — quem publica não precisa saber quem lê, e quem lê pode reprocessar o histórico inteiro sem pedir nada de volta ao produtor.

O domínio atende os quatro critérios

ADR-002 · aceita em 16/08/2026. Processo: do cadastro do animal na fazenda até o embarque para o frigorífico.

1

Ponto de decisão com regra de negócio

O embarque só é confirmado se o peso atingir a meta mínima acordada em contrato. Quem decide é a expedição, com base nesse número — não o comercial.

2

Sistema externo

O Sistema do Frigorífico recebe o romaneio e pode recusar o animal na triagem. Fora do nosso controle de deploy e sem contrato de API documentado.

3

Caminho de exceção com compensação

AnimalRejeitadoNoEmbarque: o animal permanece no lote de origem e a dieta é reavaliada — hoje implementado como saga coreografada (Parte 05).

4

Algo que valha reprocessar

PesagemRegistrada. Se o serviço cair ou o broker atrasar, o histórico precisa ser reprocessável sem duplicar leituras — sustenta a decisão de embarcar.

Quatro serviços, seis eventos, um broker

Sem módulo compartilhado: cada lado declara a própria classe do evento que consome.

PUBLISHERS



Os seis fatos de negócio

Cada evento tem contrato escrito, chave de partição definida e uma razão de existir.

EVENTO	PUBLICADO POR	CHAVE	POR QUE EXISTE
PesagemRegistrada	servico-pesagem	animalId	Histórico de peso reprocessável — sustenta lote, dieta e a decisão de embarque.
VacinacaoRegistrada	servico-vacinacao	animalId	Carteira em dia é pré-requisito de embarque na triagem do frigorífico.
LoteFormado	servico-manejo	loteId	Origem do stream de localização do lote (event sourcing, ADR-005).
LoteMovidoDePasto	servico-manejo	loteId	Atualiza a localização — o segundo fato do mesmo stream.
AnimalEmbarcadoParaAbate	servico-expedicao	animalId	O fato final do ciclo comercial — o caminho feliz. Sem consumidor hoje.
AnimalRejeitadoNoEmbarque	servico-expedicao	animalId	O caminho de exceção: dispara a saga de compensação (Parte 05).

DUAS CHAVES DE PARTIÇÃO, POR DESIGN · ADR-003

animalId para fatos de nível de animal, lotId para fatos de nível de lote. O custo aceito: a ordem causal exata ENTRE um fato de lote e um fato de animal não é garantida pelo broker — só por comparação de ocorridoEm.

02

PARTE 02

Contratos e entrega

O que o publisher promete, e o que o consumidor precisa aguentar receber.

- | Envelope CloudEvents 1.0 em modo binário
- | Chave de partição e compatibilidade BACKWARD
- | At-least-once e o consumidor idempotente

Envelope CloudEvents 1.0 em modo binário

Atributos ce_* nos cabeçalhos; o corpo carrega só o fato de negócio.

CABECALHOS · INFRAESTRUTURA

```
ce_specversion : 1.0
ce_id          : evt-vac-2026-000123
ce_source      : /fazenda-corte/vacinacao-service
ce_type        : gado.animal.vacinacao-registrada.v1
ce_time        : 2026-08-20T09:15:00Z
ce_subject     : animal/AN-004821
ce_datacontenttype : application/json
```

CORPO · NEGOCIO

```
{
  "eventoId": "evt-vac-2026-000123",
  "ocorridoEm": "2026-08-20T09:15:00Z",
  "animalId": "AN-004821",
  "pesoKg": 398.5,
  "vacina": "Febre Aftosa",
  "validade": "2027-02-20T23:59:59Z"
}
```

SEMÂNTICA DO CAMPO

validade NÃO é a data de aplicação

Essa é ocorridoEm. validade sustenta a regra “carteira em dia”. Data no passado não é evento inválido — é condição de negócio legítima.

EVOLUÇÃO DO SCHEMA

Compatibilidade BACKWARD

Schema novo lê dado escrito com schema antigo. Trocar tipo de campo existente quebra a regra e exige .v2.

BUG REAL CORRIGIDO · O CAMPO VALIDADE

Chegou declarado como java.util.Date — tipo que o JavaTimeModule não cobre, serializando um ISO-8601 “torto” (+0000 em vez de Z). Trocado para Instant nas duas pontas, enquanto ainda não havia consumidor externo.

At-least-once exige idempotência — não é opcional

Confirmar o offset depois de processar é o que caracteriza at-least-once: se o processo morre entre o commit e o ack, o evento volta.



```

@Transactional // no SERVICE, nao no listener
public boolean processar(String eventoId, ...) {
    boolean primeiraVez =
        repositorio.registrarEventoSeNovo(eventoId);
    if (!primeiraVez) return false; // duplicata
    repositorio.registrarPesagem(animalId, pesoKg, ..);
    return true;
}
  
```

Duas decisões que importam

- Dedup por eventoid, NUNCA por animalId. Duas pesagens do mesmo animal são dois fatos legítimos — deduplicar por animal apagaria a curva de peso.
- INSERT direto, sem SELECT antes: a corrida entre dois consumidores da mesma partição ficaria visível numa checagem em duas etapas.

PROVADO POR TESTE · HISTORICOPESAGEMSERVICETEST

mesmoEventoEntregueTresVezesRegistraUmaSoVez() — três entregas, um só efeito persistido. Roda em H2, sem Docker e sem o publisher no ar.

03

PARTE 03

Agregação por janela

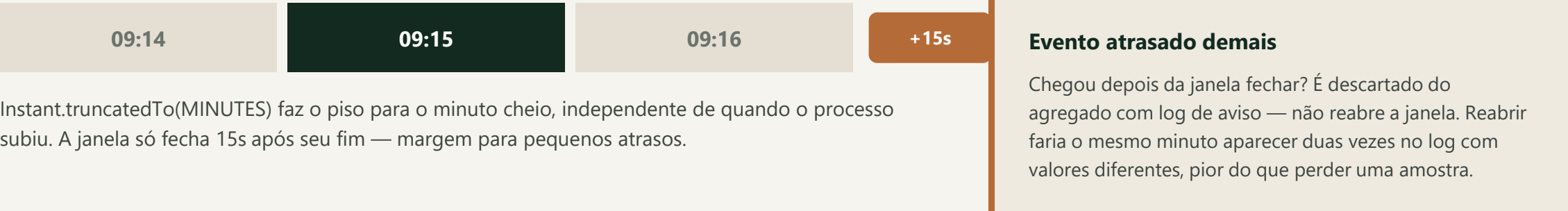
Uma pergunta de negócio respondida sobre o fluxo — e as escolhas de relógio que ela obriga.

- | Peso médio do rebanho por janela de 1 minuto
- | Event time vs. processing time
- | O que acontece com o evento atrasado e com o replay

Peso médio do rebanho por janela de 1 minuto

O número que sai é pesoMedioKg — o que zootecnia usa para decidir dieta e formação de lote. Não é “quantos eventos por minuto”, que seria métrica de infraestrutura.

JANELA ALINHADA PELO RELÓGIO · UTC, MINUTO CHEIO



Instant.truncatedTo(MINUTES) faz o piso para o minuto cheio, independente de quando o processo subiu. A janela só fecha 15s após seu fim — margem para pequenos atrasos.

DECISÃO

Relógio: hora de OCORRÊNCIA

ocorridoEm (event time), não hora de chegada. Processing time faria a janela representar “quando o consumidor processou” — propriedade da infraestrutura do dia, não do animal.

CONSEQUÊNCIA ACEITA

Replay dá o mesmo resultado?

Não necessariamente — e tudo bem. O conjunto de eventos de cada janela é determinístico (vem de ocorridoEm), mas o fechamento usa Instant.now(). Divergir só é possível num replay parcial.

POR QUE O AGREGADOR NÃO TEM ACK MANUAL

São garantias diferentes, de propósito: o histórico grava estado permanente e exige dedup + transação + ack manual; a agregação é observabilidade de negócio e o commit automático basta.

04

PARTE 04

Event sourcing do agregado Lote

O estado atual derivado do histórico de fatos — em vez de guardado e sobrescrito.

- | Por que o Lote, e não o Animal (ADR-005)
- | Event store append-only + projeção descartável
- | Reconstrução por replay, provada em teste

Feature · Event sourcing da localização do lote

ADR-005 · O agregado escolhido é o Lote, no recorte específico da sua localização de pasto.

O PROBLEMA CONCRETO QUE MOTIVOU A DECISÃO

LoteFormado e LoteMovidoDePasto eram publicados no Kafka, mas nenhum consumidor os lia. Não existia, em lugar nenhum do sistema, uma tabela ou coluna que respondesse “em qual pasto está o lote X agora” — só relendo o tópico do começo, manualmente.

ESCOLHA DO AGREGADO

Por que o Lote, e não o Animal

É o único agregado sem nenhuma outra fonte de verdade para o estado atual.

Animal já tem histórico de peso e vacinação em tabelas próprias — o problema que event sourcing resolve já está resolvido ali de outra forma. Para “onde está o lote”, não havia alternativa: o ganho é concreto, não hipotético.

ALTERNATIVA

A alternativa recusada

Uma coluna `pasto_atual_id` mutável na tabela `lote`, atualizada por `UPDATE`.

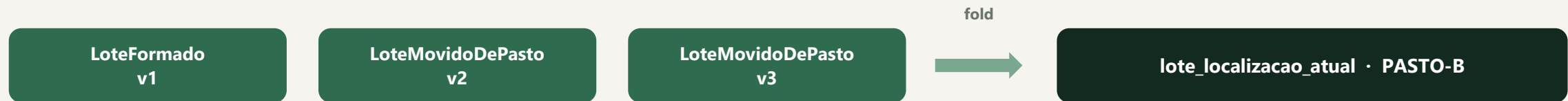
Recusada: sobrescrever apaga a única coisa confiável que se tinha — o histórico de por onde o lote passou. E se a coluna corromper, não haveria como saber o valor correto, porque nada intermediário teria sido guardado.

CONSEQUÊNCIA ACEITA

A leitura da localização deixa de ser um `SELECT` instantâneo: passa a depender de uma projeção mantida de forma assíncrona. Existe uma janela real entre “o lote foi movido” e “a projeção reflete isso”. Tolerância documentada por tela: alguns minutos — nenhuma decisão automática reage a essa consulta hoje.

Event store append-only e projeção descartável

A fonte da verdade é o stream de eventos; a projeção é um derivado que pode ser apagado e refeito.



lote_evento_store — chave primária (lote_id, versao) · append-only · uma linha por evento do stream

LOTELOCALIZACAOSERVICE

```
// o fold: mesma logica incremental e por replay
long versao = store.proximaVersao(loteId);
boolean anexado = store.anexar(loteId, versao,
    eventoId, tipo, ocorridoEm, payload);
if (!anexado)
    throw new IllegalStateException("conflito de versao");

// LoteFormado -> cria a linha, ainda sem pasto
// LoteMovido -> substitui pelo pastoDestinoId
```

CONTROLE DE CONCORRÊNCIA

A PK detecta concorrência sozinha

Para anexar o evento seguinte, o consumidor lê a versão mais alta do lote e tenta inserir versao + 1.

Se duas escritas concorrentes partirem do mesmo estado, o banco aceita a primeira e rejeita a segunda por violação de chave. Quem perde relê e tenta de novo.

Sem lock explícito e sem coluna de versão otimista separada — a própria numeração sequencial do stream já é o mecanismo.

A PROVA DE QUE A PROJEÇÃO É DESCARTÁVEL

projecaoReconstruidaPorReplayDaOMesmoResultadoQueOFoldIncremental() — o teste apaga a projeção de um lote, reconstrói lendo só o event store e verifica resultado idêntico. Na API: POST /api/lotos/{id}/localizacao/reconstruir.

05

PARTE 05

Resiliência

O que o sistema faz quando o caminho feliz não acontece — no negócio e na infraestrutura.

- | Saga coreografada: a compensação da rejeição no embarque
- | DLQ permanente em PostgreSQL
- | Reprocessamento manual, com envelope CloudEvents restaurado

Feature · servico-expedicao e o caminho de exceção

O quarto serviço fecha o ciclo comercial: publica o embarque bem-sucedido e, quando o frigorífico recusa, publica a rejeição.

CAMINHO FELIZ

AnimalEmbarcadoParaAbate

O caminho feliz — o fato final do ciclo comercial do animal.

Publicado, mas sem consumidor hoje: não existe projeção de “quais animais já embarcaram”. Está nomeado como pendência consciente, não esquecimento.

CAMINHO DE EXCEÇÃO

AnimalRejeitadoNoEmbarque

O caminho de exceção — o frigorífico recusou o animal na triagem de recebimento.

Consumido por servico-manejo (grupo manejo-rejeicao-embarque), dispara a saga de compensação descrita no próximo slide.

OS TRÊS EFEITOS DA COMPENSAÇÃO · na MESMA transação do dedup

1

Permanece no lote de origem

Nenhum fluxo deste sistema desassocia um animal do lote só por entrar em venda — “retornar” aqui é reafirmar o lote_id que ele já tem, não mover nada.

2

Dieta reavaliada

PESO_INSUFICIENTE → REFORCO_ENERGETICO; qualquer outro motivo → MANUTENCAO. Estado vigente novo em animal.dieta_atual, não histórico.

3

Venda em aberto encerrada

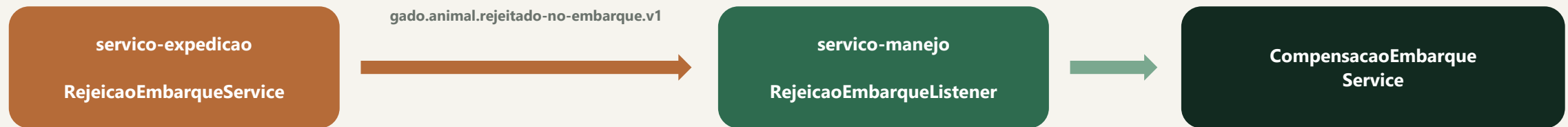
Toda venda ainda aberta que referencia aquele animalId é encerrada — a tentativa de venda que não se concretizou.

A COMPENSAÇÃO NÃO CORRIGE O PASSADO

Nenhum UPDATE ou DELETE desfaz o fato original da recusa. A compensação é o que o servico-manejo FAZ a partir dele — um novo estado vigente, não uma reversão do histórico.

Feature · Saga coreografada, não orquestrada

ADR-006 · Não existe nenhum componente central coordenando os passos: cada serviço reage ao que já aconteceu.



Nenhum dos dois serviços chama o outro diretamente, nem espera resposta síncrona.

BENEFÍCIO

O que se ganhou

Os dois serviços continuam publicáveis e testáveis de forma independente — expedição não precisa saber que manejo existe.

Se serviço-manejo estiver fora do ar quando a recusa acontece, o evento fica retido no tópico e a compensação roda quando o consumidor voltar. Não há ponto único de falha coordenando a saga.

CONSEQUÊNCIA ACEITA

O que se perdeu

Não existe lugar nenhum que enxergue “o progresso da saga como um todo” — para saber se uma compensação rodou, consulta-se o estado do manejo diretamente.

Também não há timeout nem cancelamento automático: se o evento nunca for processado, quem percebe é quem for procurar.

E SE A PRÓPRIA COMPENSAÇÃO FALHAR?

Cai na mesma evento_dlq (próximo slide). A saga fica incompleta e VISÍVEL: há uma linha dizendo exatamente que aquela recusa ainda não foi compensada. Não inventamos um segundo conceito para o mesmo problema.

Feature · DLQ permanente em PostgreSQL

Para a mensagem que NUNCA vai conseguir processar: poison message que não desserializa, ou regra de negócio que lança sempre.



Só depois dessa gravação o offset original é confirmado — a partição nunca trava, e nada é descartado em silêncio.

CONTEÚDO DA LINHA

O que fica gravado

- O payload — ou os bytes originais do fio, no caso de poison message;
- o envelope CloudEvents INTEIRO capturado dos headers, não só ce_id/ce_type;
- a exceção raiz e a posição Kafka exata (tópico/partição/offset).

Dedup por UNIQUE (origem_topico, particao, deslocamento) — a posição existe até para poison message sem ce_id aproveitável.

ALTERNATIVA RECUSADA

Por que tabela, e não tópico .dlq

O padrão canônico do Spring Kafka (DeadLetterPublishingRecoverer) foi recusado por critério, não por desconhecimento:

um tópico morto tem retenção finita e depende do broker para ser lido. A tabela é o registro literalmente permanente, consultável no banco que o serviço já opera e testável com H2, sem broker.

CONSEQUÊNCIA ACEITA

A DLQ acumula e não se auto-reprocessa. Se a gravação da própria DLQ falhar, a exceção se propaga e o offset não é confirmado — reprocessar custa CPU, mas perder em silêncio é pior.

Feature · reproprocessamento manual da DLQ

POST /api/dlq/{id}/reprocessar reenvia o payload ao tópico ORIGINAL, com os cabeçalhos restaurados — sem caminho de código separado.

```
O CAMINHO MANUAL

$ curl http://localhost:8083/api/dlq
[ { "id": 1,
    "origemTopico": "gado.animal.pesagem-registrada.v1",
    "particao": 1, "deslocamento": 884,
    "motivo": "DeserializationException" } ]

$ curl -X POST ../api/dlq/1/reprocessar
# -> reenvia ao topico original, headers restaurados
```

AUDITORIA PRESERVADA

Reprocessar não altera a linha original

evento_dlq permanece intacta para auditoria. Cada tentativa vira uma linha nova em dlq_reprocessamento — append-only, mesma lógica das demais tabelas de fato.

Reprocessar o mesmo evento várias vezes não duplica o efeito: a idempotência por eventold já protege. Reprocessamento manual é só mais uma forma de a mensagem chegar.

POR QUE O REPROCESSAMENTO É MANUAL, DE PROPÓSITO

Um agendador que tentasse reprocessar sozinho depois de N minutos não foi construído: decidir quando a causa raiz está corrigida é humano, nesta etapa. Quem corrige decide, evento a evento, que aquilo deve voltar — e apagar da DLQ é uma decisão que o sistema nem sequer expõe.

O CONJUNTO DE TESTES DA DLQ · DlqServiceTest

falha registrada fica gravada na DLQ permanente	mesma posição Kafka fica gravada uma só vez	recoverer move erro de regra com o envelope inteiro	recoverer move poison message com payload bruto	cada reproprocessamento vira linha nova sem alterar a origem
---	---	---	---	--

O que se enxerga às 3 da manhã

Três perguntas que o sistema responde hoje — e uma que ele ainda não responde.

“O consumidor de pesagem está processando, ou parou?”

Log estruturado do PesagemListener (uma linha por evento, com eventId/animalId/pesoKg) mais o consumer lag do grupo manejo no Kafka UI. Lag crescendo sem log novo = consumidor parado.

“Tem coisa acumulando na DLQ?”

GET /api/dlq — cada linha traz motivo, tópico de origem e quando aconteceu. Contagem crescente indica falha sistemática, não evento isolado.

“A localização do lote está desatualizada?”

Comparar lote_localizacao_atual.versao_da_projecao com a versão mais alta em lote_evento_store. Se a projeção está atrás, a reconstrução por replay resolve.

“Esse animal está com a vacinação em dia?”

NÃO tem sinal hoje — só consulta SQL direta. O repositório existe, o controller que exporia isso não foi construído. Listado como pendência, não esquecido.

O que a arquitetura de eventos nos ensinou

Seis lições que saíram de decisões reais — e, em dois casos, de bugs que encontramos.

At-least-once não é detalhe de infra

Escolher ack após o processamento define a obrigação do consumidor: idempotência virou requisito, provado por teste.

group.id é decisão de arquitetura

Um caractere errado transforma dois consumidores independentes em concorrentes pelas mesmas partições — bug real, corrigido.

Nem todo consumidor precisa da mesma garantia

Histórico exige transação e ack manual; agregado de observabilidade não. Escolher a garantia mais fraca é decisão, se documentada.

Event time é do negócio; processing time é da máquina

Agregar por ocorridoEm custa tratar atraso explicitamente, mas entrega um número que significa o mesmo em toda execução.

Estado derivado é mais honesto que estado sobrescrito

O event store guarda o que aconteceu; a projeção é descartável. Um UPDATE apagaria a única coisa confiável que se tinha.

Falha precisa ser visível, não silenciosa

DLQ permanente + saga coreografada: quando algo não processa, existe uma linha dizendo exatamente isso, com o evento preservado.

Cobertura da entrega, aula a aula

Cada etapa da disciplina, o que foi pedido e onde está no repositório.

02	Domínio + publisher e consumer idempotente	ADR-002 · servico-pesagem · PesagemListener · teste de idempotência
03	Contrato de evento + segundo agregador	contrato.md · contrato-pesagem.md · PesagemAgregadaPorMinutoListener
04	Chave de partição formalizada + janela	ADR-003 · duas chaves: animalId / loteId · janela de 1 min
05	Event sourcing: agregado e projeção	ADR-005 · lote_evento_store · lote_localizacao_atual · teste de replay
FINAL	Projeto final · Parte A: DLQ e reprocessamento	ADR-006 · evento_dlq · DlqRecoverer · POST /api/dlq/{id}/reprocessar
FINAL	Projeto final · Parte B: Saga de compensação	ADR-006 · servico-expedicao · CompensacaoEmbarqueService

A evidência: 26 testes, 0 falhas

mvn -f servico-manejo/pom.xml test · executado em 23/09/2026, com H2 em memória — sem Docker e sem broker no ar.

3

HistoricoPesagem...Test

Idempotência: o mesmo evento entregue três vezes produz um só efeito.

5

CompensacaoEmbarque...Test

A saga: dieta por motivo, venda encerrada, compensação idempotente.

7

DlqServiceTest

DLQ: poison message, erro de regra, dedup por posição, reprocessamento.

5

LoteLocalizacao...Test

Event sourcing: fold incremental e replay dando o mesmo resultado.

6

CadastroManejo...Test

Cadastro: fazenda, lote, animal e venda — reenvio idempotente.

O QUE OS TESTES PROVAM, E O QUE NÃO PROVAM

Provam as regras que sustentam as decisões de arquitetura: idempotência por eventold, reconstrução da projeção por replay e o comportamento da DLQ nos dois modos de falha.

Não provam a integração com o broker real — rodam com H2 e chamam os services diretamente. Teste de ponta a ponta com Kafka no ar ficou de fora do escopo, e está nomeado como tal.

ENTREGA FINAL · EQUIPE 04

Um sistema que registra o que aconteceu

Quatro serviços, seis eventos, quatro ADRs com as consequências aceitas e 26 testes verdes. Nenhum UPDATE corrige o passado: o histórico é a fonte da verdade, a projeção é descartável e a falha fica visível na DLQ em vez de sumir em silêncio.



Arquitetura de Eventos Distribuídos · PUC Minas · 2026/2

POR ONDE COMEÇAR A LEITURA

docs/arquitetura.md

ADR-002 · domínio

ADR-003 · partição

ADR-005 · event sourcing

ADR-006 · resiliência

docs/contrato*.md

docs/openapi-*.yaml

docs/entregas/aula-0*.md